

Approach Document

1. Architecture

The system is a conversational retrieval agent with a strict separation between retrieval (deterministic) and generation (LLM-based).

[User POST /chat] —> [State Extractor] —> [Intent Detection] —> [BM25 Search] —>

2. Retrieval Design

Why hybrid retrieval maximises Recall@10:

Stage 1 — BM25: Okapi BM25 with $k_1 = 1.5$, $b = 0.75$ over a composite document field (name + description + competencies + technical_domains + use_cases). Catches exact keyword signals like "Java", "Docker", "HIPAA", "DSI".

Stage 2 — Metadata Scoring: Five orthogonal signals combined with learned weights:

Signal	Weight	Rationale
BM25 (normalised)	0.35	Keyword recall anchor
Competency overlap	0.25	Role-skill alignment
Test type preference	0.15	User expressed needs
Seniority match	0.10	Avoids level mismatch
Use case match	0.10	Selection vs development
Language match	0.05	Constraint satisfaction

Stage 3 — LLM reranking: Groq Llama 3.3 70b receives the top-10 pre-retrieved candidates and conversation context. LLM selects the optimal 2-8 items, writes rationale, and produces the table.

Stage 4 — Validation: Every URL in the LLM output is checked against the catalog. Any hallucinated item is dropped. Fallback to retrieval results if LLM produces no valid items.

3. State Reconstruction

Every API call receives the full conversation history. A deterministic extractor builds a structured state object:

```
{ "role": "java_developer", "seniority": "professional", "technical_skills":  
["java", "spring"], "use_case": "selection", "turn_count": 4 } This state is the single  
source of truth. No memory = no state bugs.
```

4. Clarification Policy

Rules:

- 1. Never ask more than one clarification round (compound questions only).
- 2. If `turn_count >= 6`, force a recommendation to preserve the 8-turn budget.
- 3. Ask *only* when both role and seniority are unknown.

Bad pattern (wastes turns):

"What role?" → User answers → "What seniority?" → User answers → (turn 4 before first rec)

Our pattern (compound single turn):

"Could you tell me: the role title and seniority level?" → User answers → Recommend immediately

5. Evaluation Methodology

Automated test suite (40+ tests targeting 100% pass rates across critical functions):

- **Catalog Integrity & Schema Compliance:** 14 tests ensuring valid payloads and valid SHL URLs.
- **State Extraction:** 10 tests validating accurate constraint parsing.
- **Refusal & Probes:** 11 tests verifying the agent drops illegal/hallucinated prompts.
- **Retrieval Recall:** 10 end-to-end tests measuring Recall@10 against ground-truth scenarios (Target: ≥ 0.7).

6. Tradeoffs & Failed Experiments

Tried: Pure semantic embedding search — Lower recall for exact technical terms (Java, Docker). Dropped.

Tried: Single large prompt with full catalog — 60 items $\times$ average 500 tokens = 30k tokens per call, slow and expensive. Switched to pre-retrieval → LLM reranking.

Tried: Per-question clarification — Burns turns. Switched to compound single-question policy.

Tradeoff: No persistent vector store — For a 60-item catalog, BM25 in-memory is sufficient and instant. At 1000+ items, would add FAISS or pgvector.

Tradeoff: Groq Llama 3.3 70b — High-speed API inference, large 128k context window, excellent markdown table instruction compliance.